```
public List<String> getTypeaheadEntries() {
    List<String> entries = new ArrayList<String>();
    for(WebElement typeaheadElement : typeaheadEntries) {
        entries.add(typeaheadElement.getText());
    }
    return entries;
}
```

Well-designed page objects also accept and return data in the form of domain objects when it makes sense to do so. In many cases, domain objects allow tests to be more expressive and readable than primitive types. For example, suppose you need to verify that the featured destination list contains a deal for Singapore costing $900 (see figure 8.14). You represent destination deals using a simple Java class called `DestinationDeal`, with a destination and a price.

If you design your page object to return a list of `DestinationDeals`, you could write test code like this:

Create a domain object representing the expected destination deal.

```
DestinationDeal expectedDeal = new DestinationDeal("Singapore", 900);
assertThat(homePage.getFeaturedDestinations()).contains(expectedDeal);
```

Check that the featured destinations on the home page contain the expected deal.

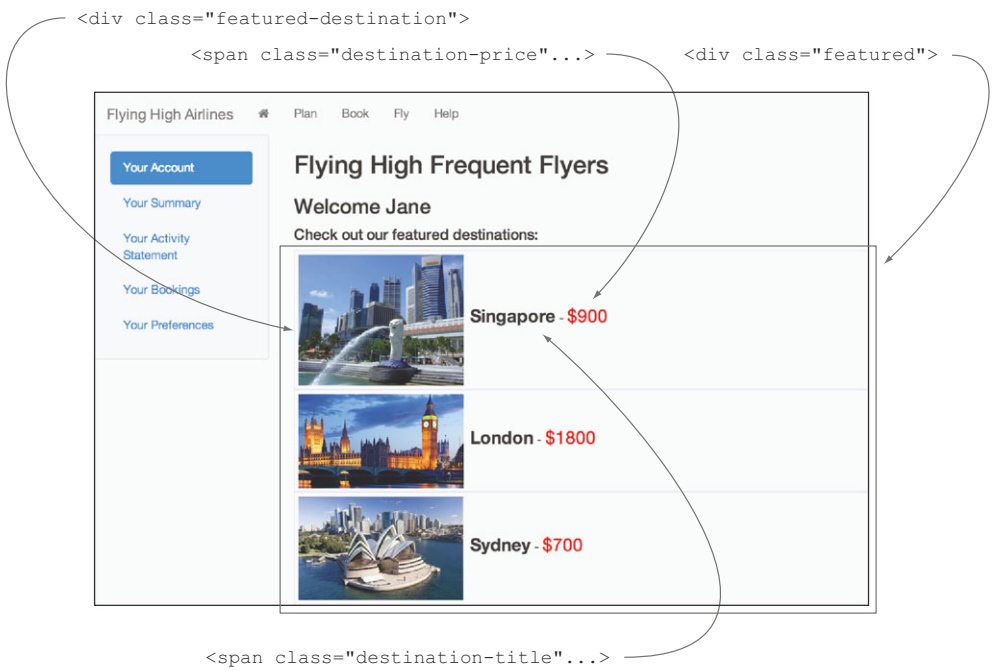


Figure 8.14 The featured destinations in this screen could be represented as a list of domain objects.